Equivariant Polynomials for Graph Neural Networks

Omri Puny*, Derek Lim*, Bobak T. Kiani*, Haggai Maron, Yaron Lipman

Introduction

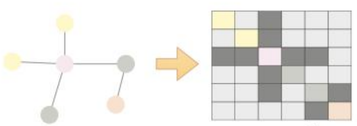
Graph Neural Networks (GNN) have limited expressive power. The Weisfeiler-Lehman (WL) hierarchy [1] has been a central tool for quantifying expressive power of GNNs. However, WL provides only a coarse evaluation of the GNN expressive power [2], and lacks direct guidance for model improvement.

Goal. 1) Propose a novel hierarchy of GNN expressive power, called *polynomial expressiveness*. 2) Use it to improve GNN expressive power.

Contributions.

Characterize equivariant graph polynomials → Define polynomial expressiveness → Relate polynomial expressiveness to tensor network contractions → Improve GNN architectures by adding polynomial features beyond their power

Graph Represented via Tensors



$G = (V, E)$ $\quad X \in \mathbb{R}^{n^2}$

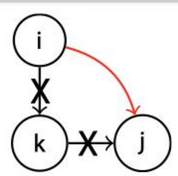
Equivariance

Let $P : V \to W$ be an arbitrary function and $H = \{h\}$ be a symmetry group. P is *H-equivariant* if:

$$h \cdot P(X) = P(h \cdot X) \quad \forall X \in V, \forall h \in H.$$

Equivariant Functions Represented via Graphs

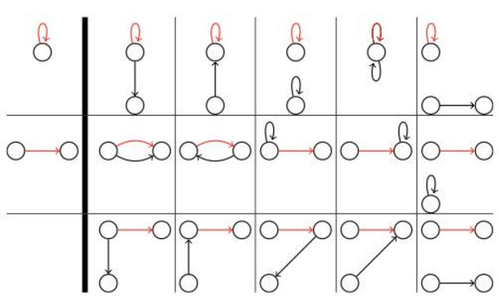
$$P(X) = X^2 \Rightarrow \begin{cases} P(X)_{ij} = \sum_{k=1}^{n} X_{ik} X_{kj} \\ P(X)_{ij} = \text{einsum}(ik, kj \to ij, X, X)_{ij} \end{cases}$$



P_H: Basis for Equivariant Graph Polynomials

Theorem. (Graph Equivariant Basis) A basis for all equivariant graph polynomials $P : \mathbb{R}^{n^2} \to \mathbb{R}^{n^2}$ of degree $\leq d$ is enumerated by direct multi-graphs H with $\leq d$ edges. The polynomial basis element corresponding to H is:

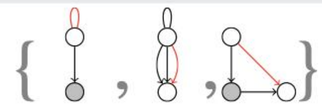
$$P_H(X) = \text{einsum}(H, \overbrace{X, \ldots, X}^{d}).$$



An enumeration of the constant and linear equivariant basis [3] using multi-graphs.

Expressive Power of Prototypical Graph Models

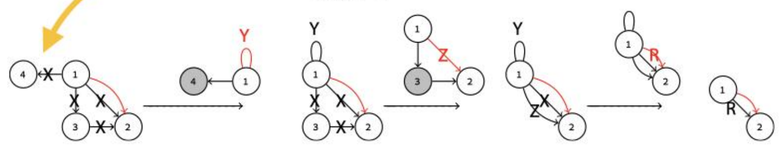
Definition. A *Prototypical Graph Model* $\mathcal{F}_{\mathcal{B}}$ is a collection of finite sequences of tensor contractions taken from a primitive bank $\mathcal{B}$.



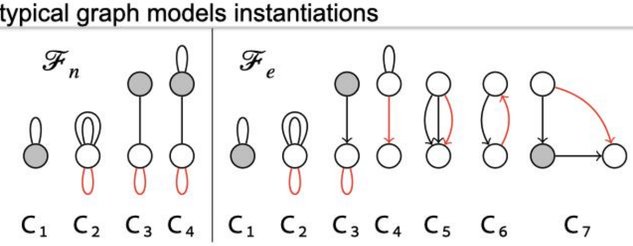
Contraction bank $\mathcal{B}$

Calculation of a polynomial via contractions from bank $\mathcal{B}$

$$P_H(\mathbf{X})_{j_1,j_2} = \sum_{j_3,j_4=1}^{n} \mathbf{X}_{j_1 j_4} \mathbf{X}_{j_1 j_3} \mathbf{X}_{j_3 j_2} \mathbf{X}_{j_1 j_2}.$$



Prototypical graph models instantiations



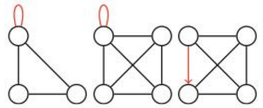
- We formulate two instantiations of prototypical graph models: $\mathcal{F}_n$ (left) for node-valued polynomials and $\mathcal{F}_e$ (right) for edge-valued and node-valued polynomials.
- We derived a linear-time algorithm (linear in polynomial degree), for deciding the computability of P_H by $\mathcal{F}_e$ and/or $\mathcal{F}_n$.

Algorithm 1 Decide if P_H is computable by $\mathcal{F}_n$ or $\mathcal{F}_e$.

Input: contraction bank $\mathcal{F} \in \{\mathcal{F}_n, \mathcal{F}_e\}$, multi-graph H
set $d = 1$ for $\mathcal{F} = \mathcal{F}_n$, or $d = 2$ for $\mathcal{F} = \mathcal{F}_e$.
set $doneContracting = false$
while not $doneContracting$ **do**
  if exists a node in $V \setminus \{a, b\}$ with $\leq d$ neighbors **then**
    contract the node using $\mathcal{F}$
  else
    $doneContracting = true$
  end if
end while
if $V \setminus \{a, b\}$ is empty **then**
  return `computable`
else
  return `non-computable`
end if

Definition A prototypical graph model $\mathcal{F}$ is d polynomial *exact* if it can compute all degree d polynomial basis elements $P_H(X)$ for every X.

- $\mathcal{F}_e$ is 4 polynomial exact (and 5 polynomial exact for node-valued polynomials). $\mathcal{F}_n$ is 2 polynomial exact.
- From WL standpoint, we found that $\mathcal{F}_e$ is equivalent to 3-WL while $\mathcal{F}_n$ is equivalent to 1-WL.



Expressive Power of GNNs

Definition. A GNN is d polynomial *expressive* if it can express all degree d polynomial basis elements P_H.

- **Message Passing Neural Network (MPNN)** [4]. The polynomial expressiveness is lower bounded by $\mathcal{F}_n$, i.e. MPNN is at least 2 polynomial expressive.
- **PPGN++.** An efficient version of the PPGN [5] architecture (augmented with linear basis elements). The polynomial expressiveness is lower bounded by $\mathcal{F}_e$, i.e. at least 4 polynomial expressive / at least 5 expressive for node-valued polynomials.

$$\mathbf{Z}^{(k+1)} = \bar{\mathrm{m}}_3\left[\bar{\mathrm{m}}_1\left[\mathbf{Z}^{(k)}, \mathbf{Z}^{(k)T}\right] \circledast \bar{\mathrm{m}}_2(\mathbf{Z}^{(k)}), \mathbf{Z}^{(k)}\right]$$

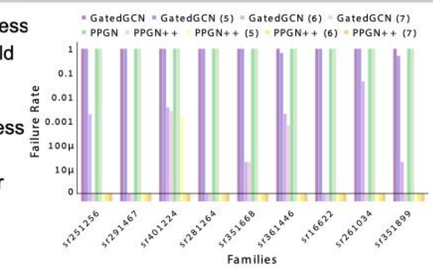
- In order to extend the polynomial expressiveness of the models, we add non-computable polynomials, determined by our algorithm, as additional input features.

	d = 3	d = 4	d = 5	d = 6	d = 7
F_n	2/8	6/18	23/49	85/144	308/446
F_e	0/18	0/53	1/174	11/604	72/2193

Number of non-computable polynomials out of all basis elements.

Results

- Increasing polynomial expressiveness achieves SOTA results on real-world datasets (ZINC and Alchemy).
- Improving polynomial expressiveness contributes to distinguishing non-isomorphic graphs (strictly stronger than 3-WL).



Model	ZINC-Full Test MAE	Alchemy Test MAE
GIN (Xu et al., 2019)	.088 ± .002	.180 ± .006
δ-2-GNN (Morris et al., 2019a)	.042 ± .003	.118 ± .001
SpeqNet (Morris et al., 2022)	-	.115 ± .001
PF-GNN (Dupty et al., 2022)	-	.111 ± .010
HIMP (Fey et al., 2020)	.036 ± .002	-
SignNet (Lim et al., 2022)	.024 ± .003	.113 ± .002
CIN (Bodnar et al., 2021a)	.022 ± .002	-
PPGN (Maron et al., 2019)	.022 ± .003	.113 ± .001
PPGN++	.022 ± .001	.111 ± .002
PPGN++ (5)	**.020 ± .001**	.110 ± .001
PPGN++ (6)	**.020 ± .001**	**.109 ± .001**

Model	Test MAE
GCN (Kipf & Welling, 2016)	.321 ± .009
GIN (Xu et al., 2019)	.163 ± .003
PNA (Corso et al., 2020)	.140 ± .006
GSN (Bouritsas et al., 2022)	.115 ± .012
PF-GNN (Dupty et al., 2022)	.122 ± .010
GIN-AK (Zhao et al., 2021)	.080 ± .001
CIN (Bodnar et al., 2021a)	.079 ± .006
SetGNN (Zhao et al., 2022)	.075 ± .003
GatedGCN (Bresson & Laurent, 2017)	.265 ± .015
GatedGCN (4)	.150 ± .005
GatedGCN (5)	.138 ± .003
GatedGCN (6)	.106 ± .003
PPGN (Maron et al., 2019)	.079 ± .005
PPGN++	.076 ± .003
PPGN++ (5)	.072 ± .005
PPGN++ (6)	**.071 ± .001**

Citations

[1] Xu et al. (2019) "How powerful are graph neural networks?" In ICLR

[2] Frasca et al. (2022) "Understanding and extending subgraph GNNs by rethinking their symmetries" In NeuriPS

[3] Maron et al. (2019) "Invariant and Equivariant Graph Networks" In ICLR

[4] Gilmer et al. (2017) "Neural message passing for quantum chemistry" In ICML

[5] Maron et al. (2019) "Provably powerful graph networks" In NeuriPS